Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF CSE

PROJECT ABSTRACT SUBMISSION

COURSE: COMPILER DESIGN (23CS2235F)

A.Y. 2026 - 2027

TITLE OF THE PROJECT	AI-Based Syntax Error Correction TooL	
S. No.	**University ID**	**Name**
1	2420030540	Allipuram Akhil
2	2420030563	B. Harneesh Yadav
3	2420030639	A.Dhanush
Name of the Guide		Uma Shankari

Abstract

The proposed project, AI-Based Syntax Error Detection, Correction and Code Optimization Tool, aims to develop an intelligent compiler-based application that analyzes source code, identifies lexical and syntax errors, suggests appropriate corrections, generates intermediate code, and performs code optimization. The system incorporates all major phases of Compiler Design, including lexical analysis, syntax analysis, syntax-directed translation, intermediate code generation, code optimization, and target code generation. Initially, the source program is processed by a lexical analyzer, which identifies tokens using regular expressions and finite automata and removes unnecessary whitespaces and comments. The identified tokens are passed to the syntax analyzer, where Context-Free Grammars (CFG), derivations, parse trees, and parsing techniques such as recursive descent, predictive parsing, LL(1), shift-reduce, SLR, CLR, and LALR are applied to validate the grammatical structure of the source code. The system detects errors such as invalid tokens, missing semicolons, unmatched brackets, incorrect expressions, and grammatical inconsistencies. AI-assisted correction techniques are then used to suggest suitable corrections and explain the detected errors.After successful syntax analysis, the system applies Syntax-Directed Translation to construct syntax trees and generate intermediate representations such as Abstract Syntax Trees (AST), Polish notation, and Three-Address Code (TAC). The generated intermediate code is further analyzed through basic blocks, flow graphs, Directed Acyclic Graphs (DAG), and global data flow analysis to identify optimization opportunities. Code optimization techniques are applied to improve execution efficiency by eliminating redundant computations and unnecessary instructions. Finally, the optimized intermediate representation is passed to the code generation phase, where target code is produced using appropriate object code forms, register allocation, register assignment, and peephole optimization.The application provides an interactive interface to display lexical tokens, parsing results, syntax trees, detected errors, suggested corrections, intermediate code, optimization results, and generated target code. The project demonstrates the practical implementation of compiler construction concepts while integrating AI-assisted error correction to improve code quality, reduce debugging effort, and help programmers understand the internal working of a compiler.

Signature of the Guide HOD